АСИНХРОННЫЙ ОБМЕН СООБЩЕНИЯМИ

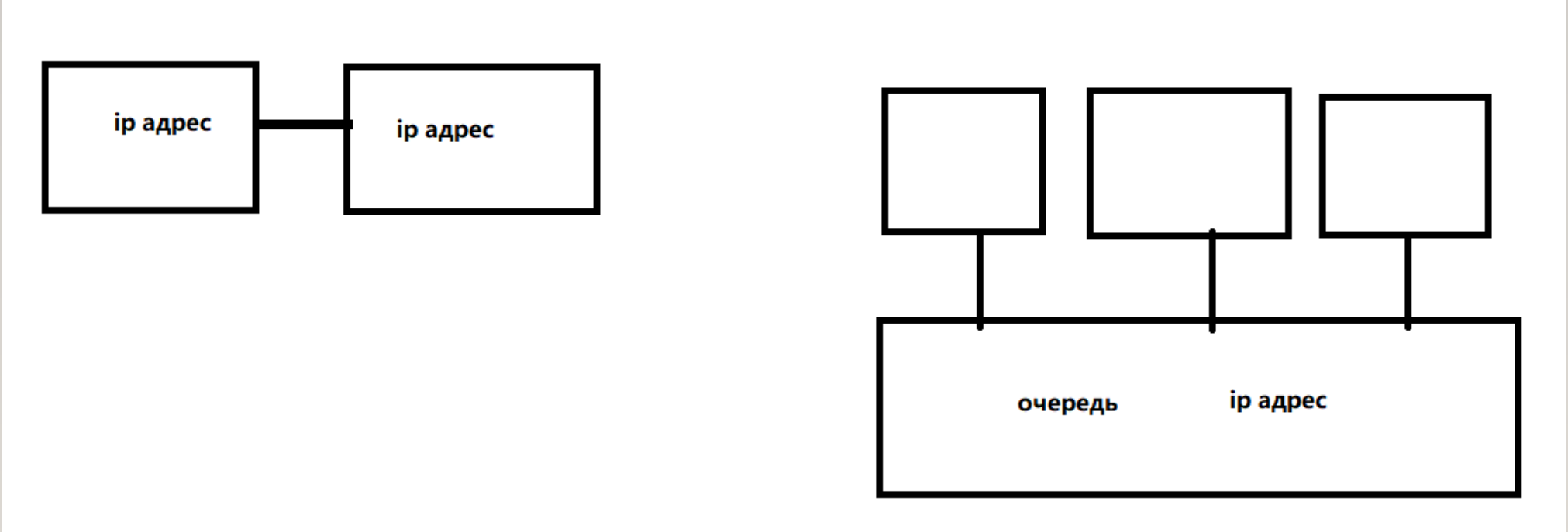
RABBITMQ И KAFKA

В КАКИХ СЛУЧАЯХ НУЖНО ИСПОЛЬЗОВАТЬ АСИНХРОННЫЙ ОБМЕН СООБЩЕНИЯМИ ЧЕРЕЗ ОЧЕРЕДЬ?

- 1 Разделение компонентов системы
- 2 Обработка долгих задач
- 3 Повышение устойчивости к сбоям
- 4 Регулирование нагрузки
- 5 Поддержка событийной архитектуры
- 6 Обработка данных в реальном времени
- 7 Масштабируемость
- 8 Интеграция различных систем
- 9 Гибкость в обработке сообщений
- 10 Аудит и трассировка

- Электронная коммерция: Обработка заказов, уведомления о статусе доставки.
- Финансовые приложения: Обработка транзакций, уведомления о подозрительной активности.
- Системы мониторинга: Сбор и анализ данных с сенсоров и устройств IoT.

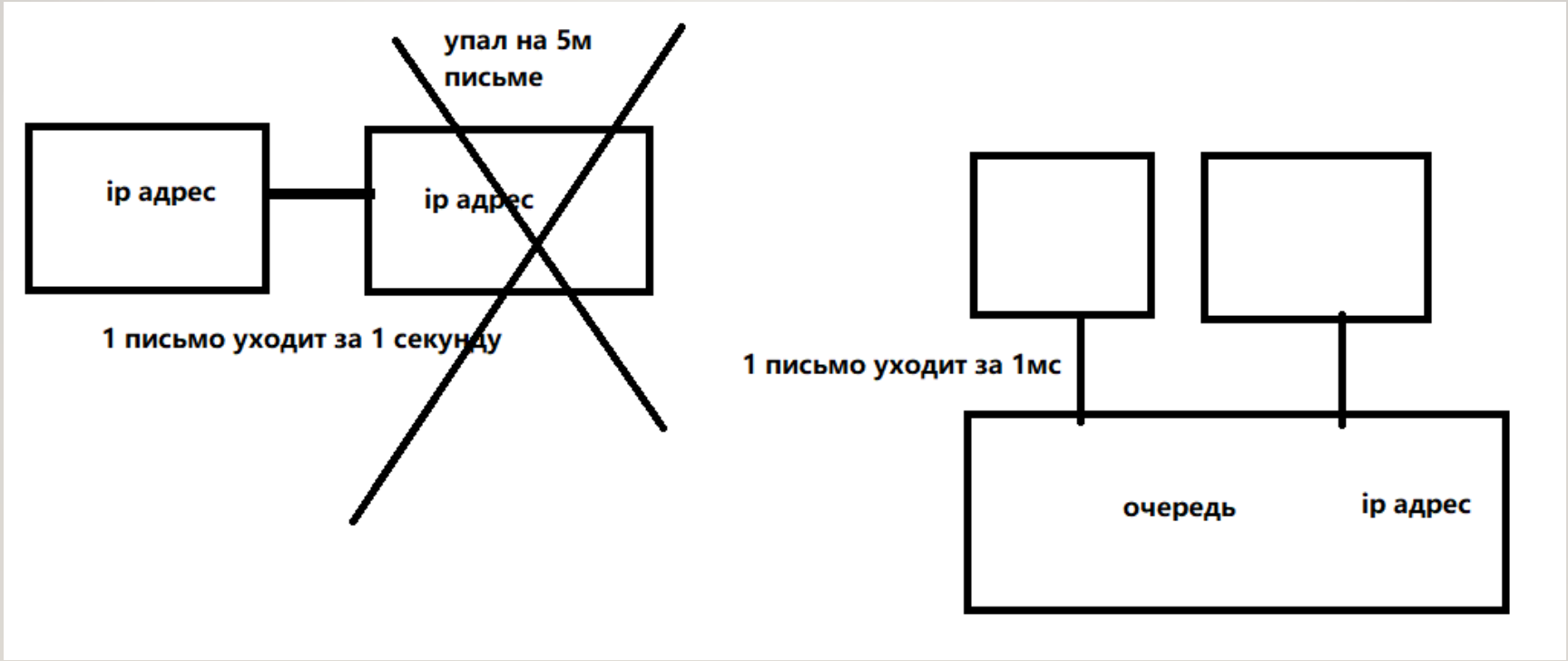
РАЗДЕЛЕНИЕ КОМПОНЕНТОВ СИСТЕМЫ



ОБРАБОТКА ДОЛГИХ ЗАДАЧ



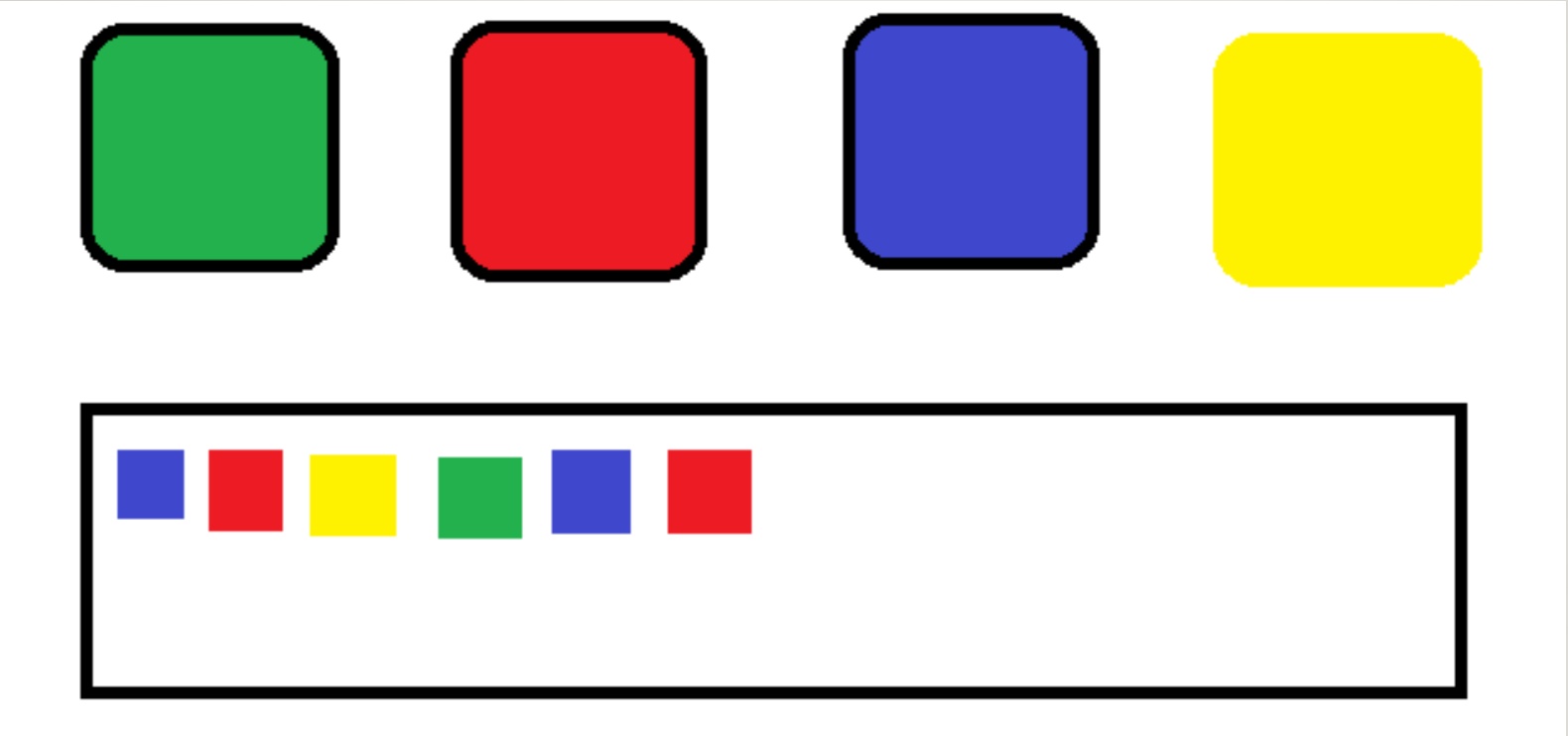
УСТОЙЧИВОСТЬ К СБОЯМ



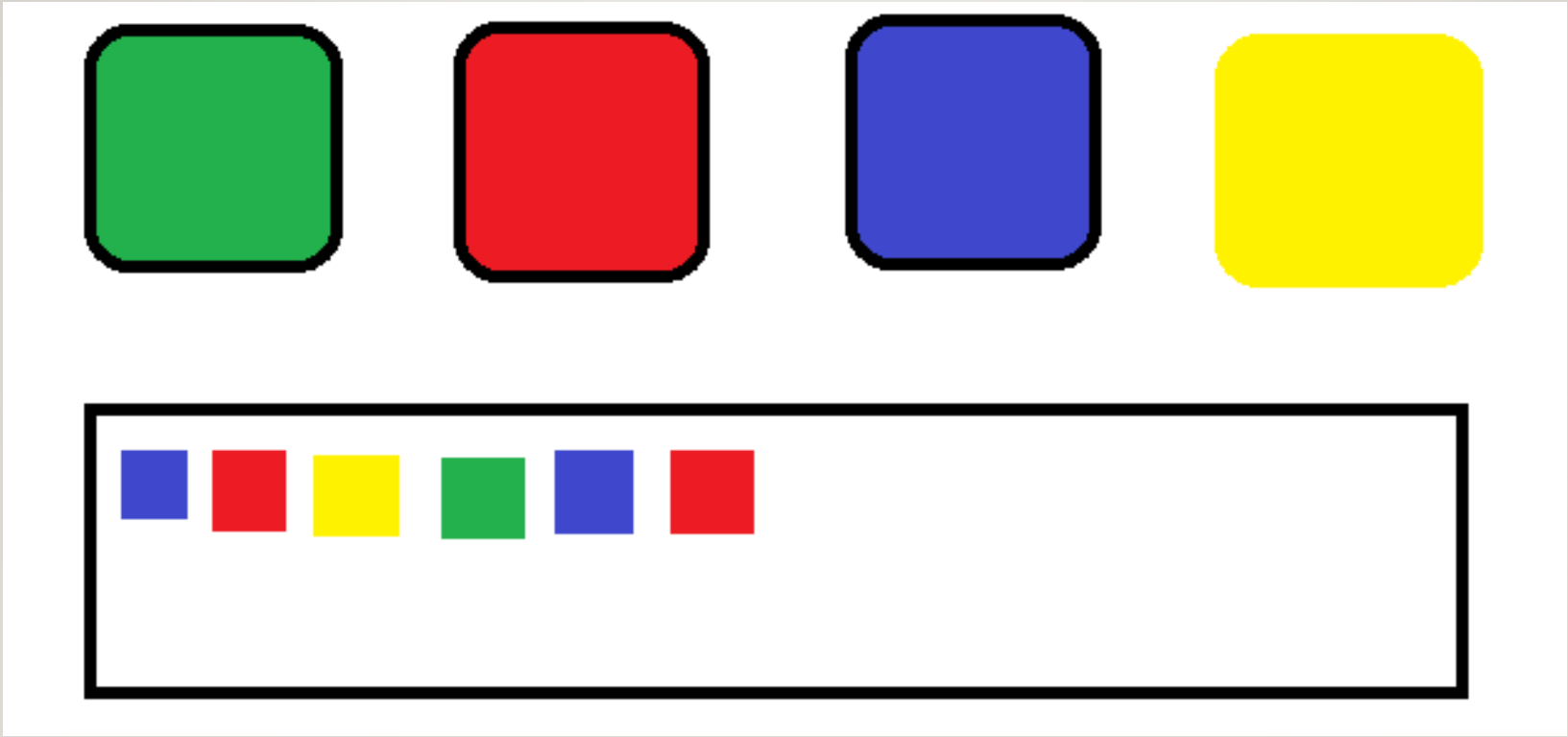
РЕГУЛИРОВАНИЕ НАГРУЗКИ



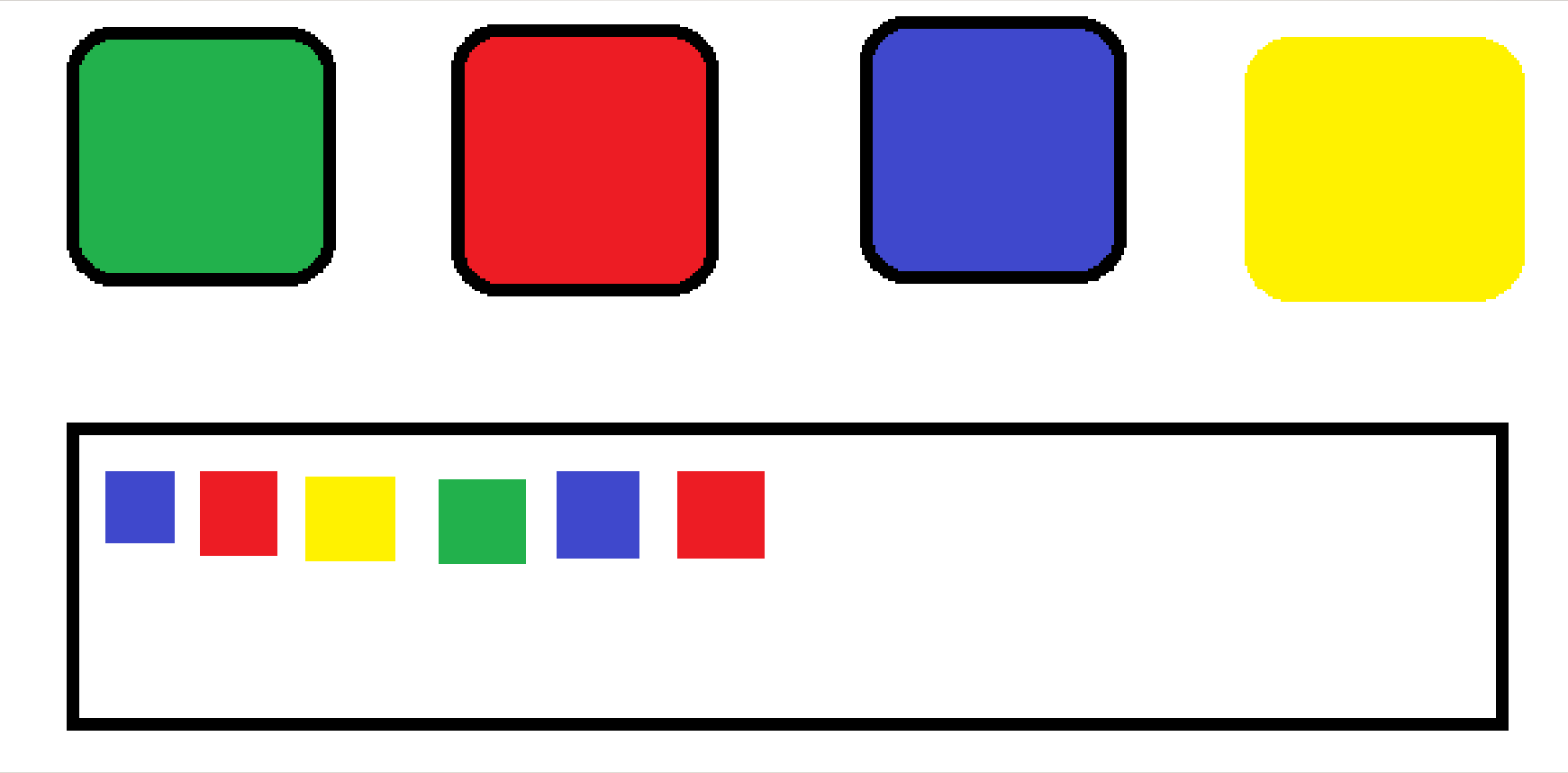
ПОДДЕРЖКА СОБЫТИЙНОЙ АРХИТЕКТУРЫ



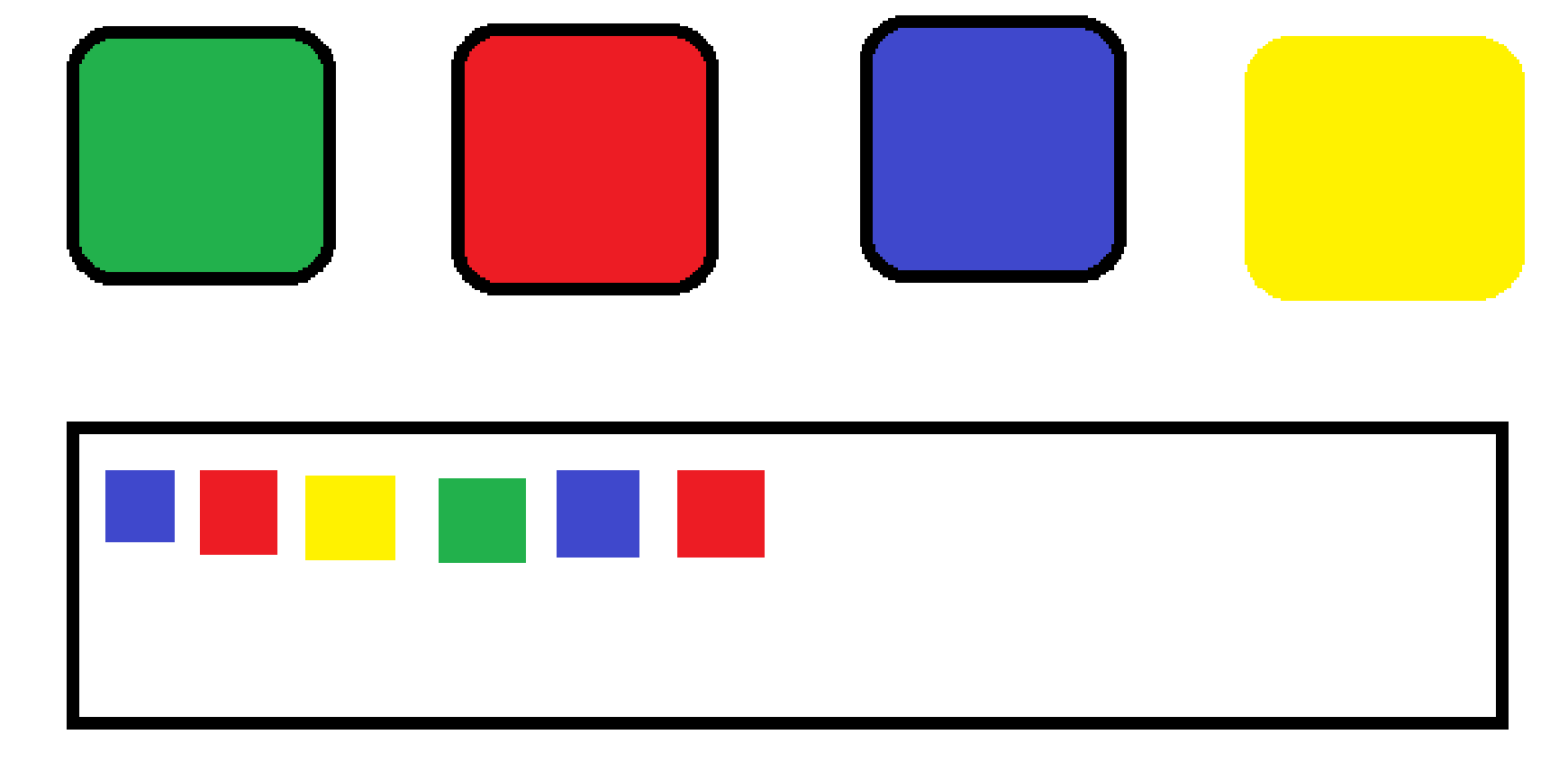
ОБРАБОТКА ДАННЫХ В РЕАЛЬНОМ ВРЕМЕНИ



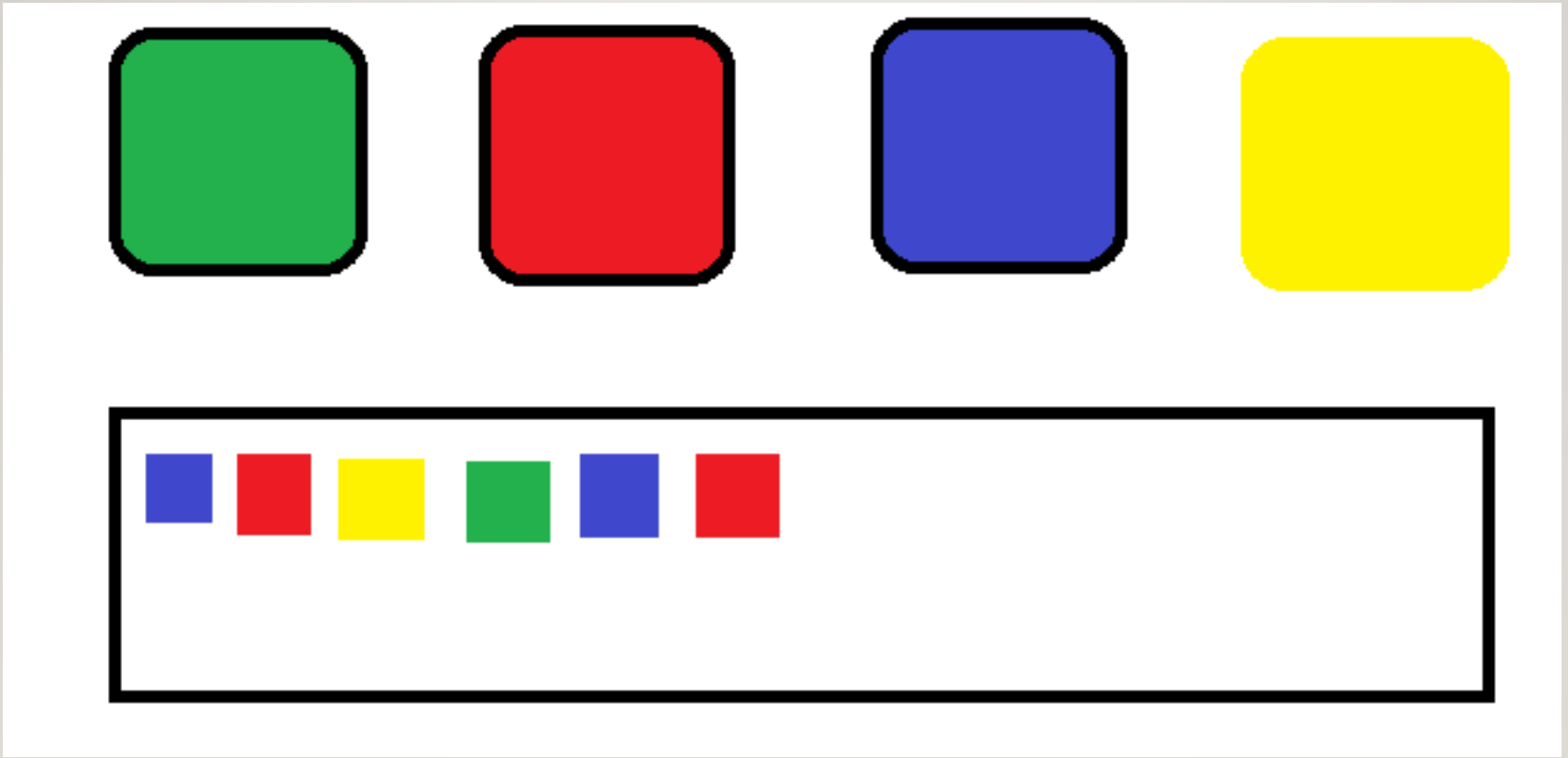
МАСШТАБИРУЕМОСТЬ



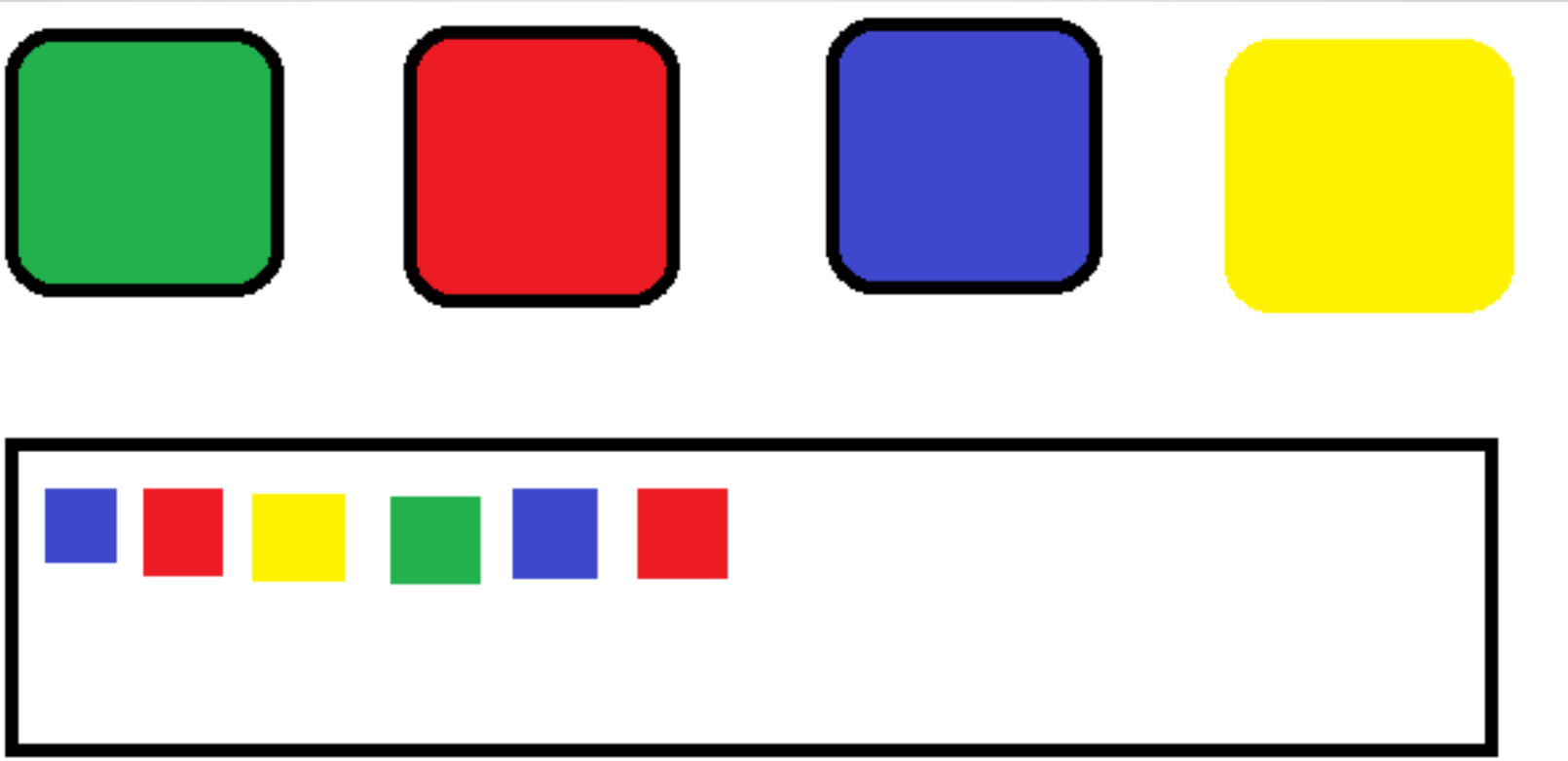
ИНТЕГРАЦИЯ РАЗЛИЧНЫХ СИСТЕМ



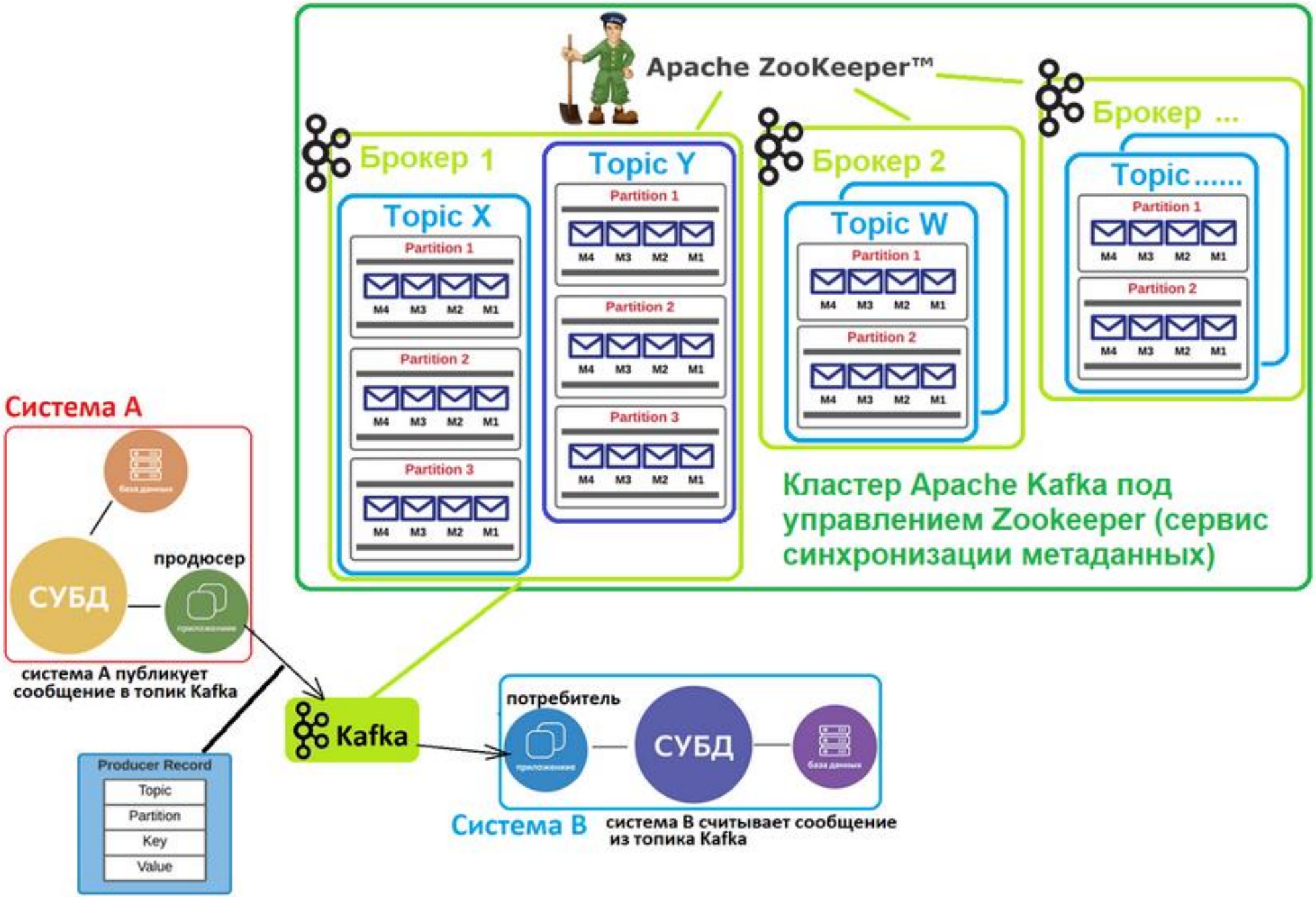
ГИБКОСТЬ В ОБРАБОТКЕ СООБЩЕНИЙ



АУДИТ И ТРАССИРОВКА

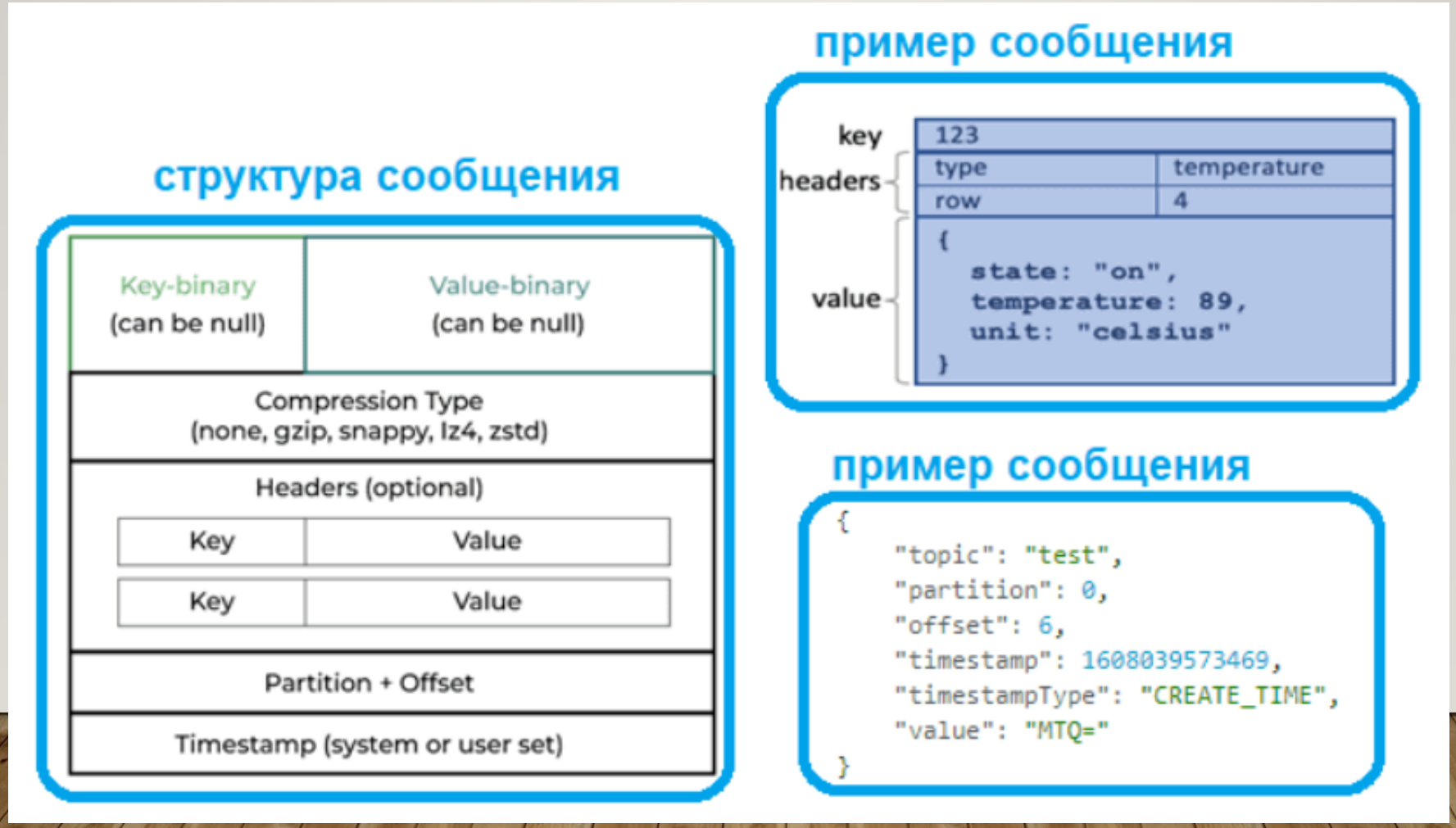


KAFKA


Apache ZooKeeper™
Брокер 1
Topic X
Partition 1
Partition 2
Partition 3
Topic Y
Partition 1
Partition 2
Partition 3
Брокер 2
Topic W
Partition 1
Partition 2
Брокер ...
Topic.....
Partition 1
Partition 2
M4 M3 M2 M1
Кластер Apache Kafka под управлением Zookeeper (сервис синхронизации метаданных)
Система А
СУБД
продюсер
система А публикует сообщение в топик Kafka
Kafka
Producer Record
Topic
Partition
Key
Value
потребитель
СУБД
Система В
система В считывает сообщение из топика Kafka

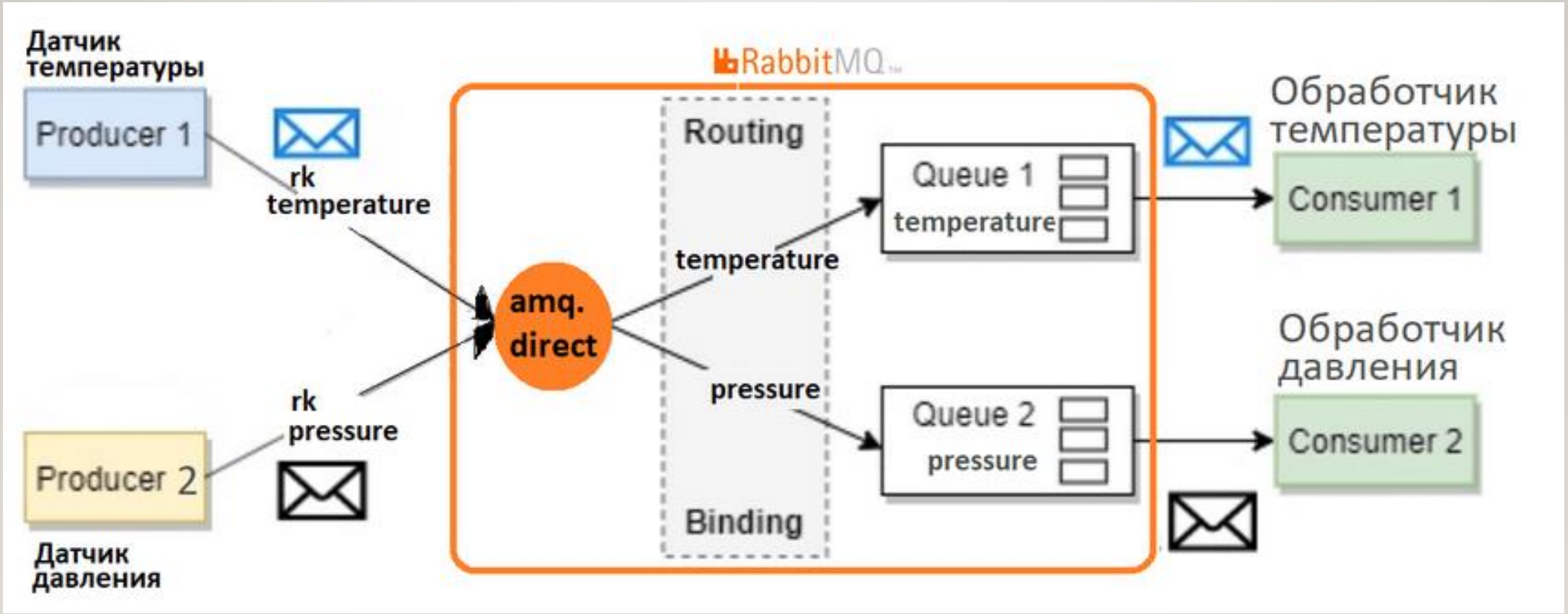
СТРУКТУРА СООБЩЕНИЯ

https://babok-school.ru/blog/kafka-as-integration-tool-overview-for-analyst/



RABBIT MQ

https://babok-school.ru/blog/rabbitmq-for-analyst/



ОБМЕННИКИ RABBIT MQ

Тип обменника	Принцип работы	Сценарий применения
Direct	Прямая отправка сообщений в одну или несколько очередей с совпадающим значением ключа маршрутизации	Когда есть точно известный ключ, по значению которого отдельные приложения-потребители должны получить подходящие сообщения
Fanout	Все сообщения отправляются во все очереди независимо от ключа маршрутизации	Когда все приложения-потребители должны быстро получать все сообщения
Headers	Маршрутизация по нескольким атрибутам, заданным в заголовке сообщения. Ключ маршрутизации игнорируется	Когда правила маршрутизации сообщения в очереди сложнее, чем просто по ключу, например, формат данных, комбинация полей и пр.
Topic	Сообщение отправляется в конкретные очереди по значению ключа маршрутизации, заданного по шаблону	Когда ключ маршрутизации сложный и поток сообщений надо разделить по разным приложениям-потребителям

АРХИТЕКТУРА

RabbitMQ	Kafka
Основан на модели очередей сообщений (message queue). Сообщения отправляются в обменники (exchanges), которые маршрутизируют их в очереди (queues) на основе ключей маршрутизации. Поддерживает различные паттерны обмена сообщениями, включая точка-точка и публикация-подписка.	Основан на концепции потоков данных (streaming) и хранилище журналов (log storage). Сообщения организованы в темах (topics), которые представляют собой последовательности записей. Сообщения сохраняются в порядке их получения и могут быть прочитаны несколькими консюмерами одновременно.

МОДЕЛЬ ДОСТАВКИ СООБЩЕНИЙ

RabbitMQ	Kafka
Поддерживает как "at-most-once" (сообщение может быть потеряно), так и "at-least-once" (сообщение будет обработано, даже если это приведет к дублированию) семантики доставки. Использует подтверждения (acknowledgments) для управления доставкой сообщений.	Предоставляет "at-least-once" и "exactly-once" (гарантия, что сообщение будет обработано ровно один раз) семантики. Сообщения сохраняются в журнале и могут быть повторно прочитаны, что упрощает обработку ошибок.

ХРАНЕНИЕ СООБЩЕНИЙ

RabbitMQ	Kafka
Сообщения обычно удаляются из очереди после их обработки, хотя можно настроить долговременное хранение для сообщений. Основной акцент на быструю и надежную доставку сообщений.	Сообщения хранятся в журналах на диске и могут храниться в течение заданного времени или до достижения определенного размера. Позволяет повторное чтение сообщений, что подходит для анализа и обработки потоков данных.

МАСШТАБИРУЕМОСТЬ

RabbitMQ	Kafka
Масштабируется вертикально (добавление ресурсов на узел) и горизонтально (кластеризация), но может быть сложнее масштабировать при очень высоких нагрузках.	Разработан для горизонтальной масштабируемости и может обрабатывать миллионы сообщений в секунду. Легко добавлять новые брокеры в кластер без значительных изменений в конфигурации.

ПРОИЗВОДИТЕЛЬНОСТЬ

RabbitMQ	Kafka
Отлично подходит для сценариев с низкой до средней нагрузкой и обеспечивает хорошую производительность для большинства приложений. Может иметь накладные расходы на обработку подтверждений и маршрутизацию.	Спроектирован для высокой производительности и может обрабатывать большие объемы данных. Эффективен для сценариев, требующих обработки больших потоков данных в реальном времени.

СЦЕНАРИИ ИСПОЛЬЗОВАНИЯ

RabbitMQ	Kafka
Идеален для традиционных приложений, требующих надежной доставки сообщений, таких как системы обработки задач, интеграция микросервисов и распределенные системы	Подходит для обработки потоков данных, анализа в реальном времени, систем мониторинга и событийной архитектуры, где требуется высокая пропускная способность и возможность повторного чтения данных.

МИНИ ВЫВОД

- Выбор между RabbitMQ и Kafka зависит от конкретных требований вашего приложения. Если вам нужна система для обработки задач и надежной доставки сообщений с поддержкой различных паттернов, RabbitMQ может быть лучшим выбором. Если же вам необходимо обрабатывать большие объемы данных в реальном времени и требуется высокая производительность, Kafka будет предпочтительнее.

ПРИМЕР

- https://habr.com/ru/articles/782234/